

Real-Time Video Processing using MPI, OpenMP, and CUDA

A Project Component for
Parallel and Distributing Computing (UCS645)

By

Sr	Name	Roll No
1	Subham Kumar	102303988
2	Jasjot Singh	102483032
3	Aditya Singh	102483007

Under the guidance of
Dr. Saif Nalband
(Assistant Professor, DCSE)



THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY
(DEEMED TO BE UNIVERSITY)

PATIALA - 147004

MAY, 2026

Table of Contents

1	Introduction	1
1.1	Background	1
1.2	Introduction to Problem Statement	2
2	Literature Review	3
3	Research Gaps	5
4	Problem Formulation	6
5	Objectives	7
6	Methodology	8
6.1	Stage 1: Motion Blur Detection (OpenMP)	8
6.2	Stage 2: Gaussian Blur (CUDA)	8
6.3	Stage 3: Background Subtraction (CUDA)	8
6.4	Stage 4: Sobel Edge Detection (CUDA)	9
6.5	MPI Frame Distribution	9
6.6	Pipeline Algorithm	10
7	System Design	11
7.1	File Structure and Module Responsibilities	11
7.2	Key Design Decisions	11
8	Experimental Configuration	13
9	Results and Discussion	15
9.1	Throughput Analysis	15
9.2	Effect of MPI Process Count on GPU Performance	16
9.3	Amdahl's Law Analysis	16
10	Performance Visualization	18
10.1	Execution Time	18

10.2 Speedup	18
10.3 Efficiency	18
11 Conclusion	19
References	20

List of Figures

1	Throughput in frames per second across all parallel configurations	15
2	Measured speedup vs number of MPI processes for CPU and GPU versions compared to ideal linear scaling	16
3	Parallel efficiency vs number of MPI processes for CPU and GPU versions	17

List of Tables

1	Summary of Literature Review	3
2	Hardware and Software Configuration	13
3	Parallel Execution Configurations Tested	13

1 Introduction

1.1 Background

Video surveillance systems, autonomous vehicles, and medical imaging platforms all share one fundamental requirement: they must process high-resolution visual data faster than it arrives. A standard 1080p camera operating at 30 frames per second generates roughly 3 gigabytes of raw data per minute. Sequential processing pipelines running on a single CPU core simply cannot keep pace with this volume, leading to dropped frames, increased latency, and degraded system reliability in time-critical deployments.

The solution lies in parallel computing. Modern hardware offers three distinct parallelism paradigms: distributed memory parallelism via the Message Passing Interface (MPI), shared memory thread-level parallelism via OpenMP, and massively parallel GPU execution via CUDA [1]. Each paradigm targets a different level of the hardware hierarchy, and combining all three into a hybrid architecture allows a system to extract maximum throughput from available resources.

This project implements a four-stage real-time video preprocessing pipeline designed around this hybrid architecture. The pipeline performs motion blur detection, Gaussian noise removal, background subtraction, and Sobel edge detection on every frame of a 1080p surveillance video, demonstrating substantial throughput improvement over a sequential baseline.

1.2 Introduction to Problem Statement

Consider a highway surveillance camera recording 1080p footage continuously. Before any intelligent analysis can occur, the raw video must pass through a preprocessing stage: poor quality frames must be identified and discarded, sensor noise must be removed, moving objects must be isolated from the static background, and structural features such as vehicle boundaries must be extracted. This preprocessing pipeline is computationally intensive. For a one-minute 1080p clip containing 3921 frames, a sequential implementation requires over 440 seconds of processing time, achieving less than 9 frames per second. This is far below the 30 FPS threshold required for real-time operation.

The challenge is therefore to parallelize this pipeline across available hardware so that processing throughput meets or exceeds the camera's capture rate. This requires careful mapping of pipeline stages to appropriate parallelism primitives: frame-level distribution across MPI processes, row-level parallelism within each frame using OpenMP threads, and pixel-level parallelism on the GPU via CUDA kernels.

2 Literature Review

Table 1: Summary of Literature Review

References	Concepts Highlighted	Technique(s) Used	Significance / Proposal	Limitations
[1]	CUDA GPU parallelism; memory hierarchy; thread occupancy	Massively parallel thread execution on GPU	Establishes the theoretical foundation for GPU-accelerated image processing; motivates pixel-level parallelism for convolution workloads	GPU-centric treatment; MPI and OpenMP hybrid CPU parallelism not covered
[2]	CUDA kernel design; memory coalescing; shared memory optimization	CUDA C programming patterns	Provides implementation-level guidance for writing efficient CUDA kernels for image filtering operations	Focuses on single-GPU execution; distributed multi-process scenarios not addressed

References	Concepts Highlighted	Technique(s) Used	Significance / Proposal	Limitations
[3]	MPI point-to-point communication; collective operations; domain decomposition	Message Passing Interface; distributed memory parallelism	Provides the theoretical basis for distributing image strips across MPI processes and collecting results via gather operations	Theoretical treatment; GPU integration not discussed
[4]	Real-time computer vision; image filtering; video capture and writing	OpenCV C++ API; Sobel and Gaussian operations	Establishes Gaussian blur and Sobel edge detection as standard pre-processing steps in production computer vision pipelines	Library-level abstraction; low-level parallel implementation details not exposed

3 Research Gaps

A review of existing literature reveals the following gaps that this project directly addresses:

- ***Absence of three-way hybrid parallelism in video preprocessing pipelines:*** Most published work targets either GPU-only acceleration [1] or CPU-only OpenMP parallelism [3]. A unified pipeline combining MPI for frame distribution, OpenMP for row-level parallelism, and CUDA for pixel-level GPU acceleration has not been systematically benchmarked in the context of real-time video preprocessing.
- ***Lack of integrated multi-stage pipeline analysis:*** Existing work typically benchmarks individual operations such as Gaussian blur or edge detection in isolation. No single reference integrates motion blur detection, noise removal, background subtraction, and edge detection into one parallel pipeline with end-to-end throughput measurement.
- ***Limited analysis of MPI overhead on single-node GPU systems:*** The interaction between MPI communication cost and CUDA kernel execution time on a single workstation has not been thoroughly characterized. This project demonstrates that MPI introduces significant overhead when all processes share one GPU, an important practical finding for system designers.

4 Problem Formulation

The core computational problem is the convolution of each video frame with a two-dimensional Gaussian kernel. For a grayscale image of width W and height H , the Gaussian blur output at pixel (x, y) is defined as:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

For a discrete 5×5 kernel, each output pixel requires 25 multiply-accumulate operations. Across all pixels of a 1080p frame:

$$\text{Operations per frame} = 1920 \times 1080 \times 25 = 51,840,000$$

At 3921 frames, the total operation count for the Gaussian stage alone exceeds 2×10^{11} floating-point operations. This computational volume provides clear motivation for parallelization.

Background subtraction computes the absolute pixel difference between consecutive blurred frames:

$$B(x, y) = |F_t(x, y) - F_{t-1}(x, y)|$$

Sobel edge detection computes the gradient magnitude from horizontal and vertical derivative kernels K_x and K_y :

$$E(x, y) = \sqrt{(F * K_x)^2 + (F * K_y)^2}$$

All three operations share the property of pixel-level independence: each output pixel depends only on a local neighborhood, making the entire pipeline embarrassingly parallel.

5 Objectives

- To implement a four-stage real-time video preprocessing pipeline comprising motion blur detection, Gaussian noise removal, background subtraction, and Sobel edge detection on 1080p surveillance video.
- To parallelize the pipeline using a three-level hybrid architecture: MPI for inter-process frame strip distribution, OpenMP for intra-process row-level parallelism, and CUDA for pixel-level GPU acceleration.
- To measure wall-clock throughput in frames per second across all parallelization configurations and characterize speedup relative to a sequential baseline.
- To analyze the interaction between MPI communication overhead and CUDA execution time, and to identify the optimal process configuration for single-node GPU deployment.
- To produce output videos demonstrating the visual effect of each pipeline stage on real 1080p surveillance footage.

6 Methodology

6.1 Stage 1: Motion Blur Detection (OpenMP)

Before committing computational resources to a frame, each MPI process evaluates the sharpness of its assigned strip using the Laplacian variance metric. A 3×3 Laplacian kernel is convolved with the strip, and the variance of the response is computed. Strips with variance below a threshold of 100.0 are flagged as blurry and bypass the remaining three stages. This mirrors the quality filtering used in production surveillance systems to discard frames degraded by camera shake or fast motion. The row-wise loop is parallelized using `#pragma omp parallel for`.

6.2 Stage 2: Gaussian Blur (CUDA)

A hardcoded 5×5 Gaussian kernel stored in CUDA `__constant__` memory is applied to each pixel of the strip. Each CUDA thread is mapped to exactly one output pixel. Boundary pixels are handled by nearest-edge clamping. The discrete kernel is:

$$K = \frac{1}{256} \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix} \quad (1)$$

6.3 Stage 3: Background Subtraction (CUDA)

Each CUDA thread computes the absolute difference between the current blurred pixel and the corresponding pixel from the previous frame strip. Moving objects produce bright output pixels while the static background maps to near zero. The previous frame buffer is updated after each subtraction, enabling motion detection without any machine learning model.

6.4 Stage 4: Sobel Edge Detection (CUDA)

Horizontal and vertical Sobel kernels stored in `__constant__` memory are applied to the blurred frame. Each thread computes the gradient magnitude:

$$E = \sqrt{S_x^2 + S_y^2}$$

The result highlights object boundaries and structural features in the scene, directly usable as input to downstream recognition models.

6.5 MPI Frame Distribution

Rank 0 reads each frame from the input video using OpenCV `VideoCapture`, converts it to grayscale, and divides it into horizontal strips. Each strip is dispatched to its assigned MPI process via `MPI_Send`. After processing, each rank returns its result strip to rank 0, which stitches the strips and writes the output frame.

6.6 Pipeline Algorithm

Algorithm 1 Hybrid Parallel Video Processing Pipeline

```
1: MPI_Init; get rank, size
2: Rank 0: open VideoCapture(input.mp4), broadcast width, height
3: for each frame do
4:   Rank 0: read frame, convert to grayscale, split into strips
5:   Rank 0: MPI_Send each strip to assigned rank
6:   Each rank: receive strip via MPI_Recv
7:   Stage 1: compute laplacian_variance with OpenMP
8:   if variance  $\geq 100.0$  then
9:     Stage 2: gpu_gaussian_blur (CUDA kernel)
10:    Stage 3: bg_sub_kernel (CUDA kernel)
11:    Stage 4: sobel_kernel (CUDA kernel)
12:   end if
13:   Each rank: MPI_Send processed strip to rank 0
14:   Rank 0: stitch strips, overlay FPS, write to VideoWriter
15: end for
16: Print total frames, total time, average FPS
17: MPI_Finalize
```

7 System Design

7.1 File Structure and Module Responsibilities

The project is organized into the following source files:

- `src/main.cpp` — MPI coordinator and pipeline orchestrator. Handles VideoCapture, frame splitting, MPI communication, strip stitching, and VideoWriter output.
- `src/cpu_blur.c` / `cpu_blur.h` — OpenMP-parallelized Gaussian blur and Laplacian variance functions for the CPU execution path.
- `src/gpu_blur.cu` / `gpu_blur.h` — CUDA kernel for Gaussian blur with persistent device memory allocation across frames.
- `src/gpu_pipeline.cu` / `gpu_pipeline.h` — CUDA kernels for background subtraction and Sobel edge detection, exposed with `extern "C"` linkage for `mpicxx` compatibility.
- `src/sequential.cpp` — Single-threaded baseline implementation of all four stages for benchmarking.
- `Makefile` — Builds `blur_cpu`, `blur_gpu`, and `blur_sequential` targets using `mpicxx` and `nvcc`.

7.2 Key Design Decisions

Separation of CUDA and MPI compilation: CUDA kernels are isolated in `.cu` files compiled with `nvcc` into object files, which are then linked with the MPI-compiled `main.cpp`. This avoids the incompatibility between `nvcc` and `mpicxx` when parsing CUDA syntax in the same translation unit.

Persistent GPU memory allocation: Device buffers for all CUDA kernels are allocated once before the frame loop using dedicated `gpu_alloc()` functions and freed after the loop. This eliminates per-frame `cudaMalloc/cudaFree` overhead which was identified as a significant performance bottleneck during initial benchmarking.

Constant memory for kernels: The 5×5 Gaussian kernel and 3×3 Sobel kernels are stored in CUDA `__constant__` memory, enabling broadcast reads across all threads in a warp with zero additional memory transaction overhead.

8 Experimental Configuration

This project does not use an externally sourced dataset. The input is a one-minute 1080p store surveillance video containing 3921 frames, depicting pedestrians moving through a retail store aisle. This provides a realistic mix of static background and moving foreground objects suitable for all four pipeline stages.

Table 2: Hardware and Software Configuration

Parameter	Value	Notes
Input video	store-aisle-detection.mp4	1080p surveillance footage
Total frames	3921	Approximately 60 seconds
GPU	NVIDIA GeForce GTX 1650	4GB VRAM, sm_75, WDDM via WSL2
CUDA version	11.5	Ubuntu 22.04 WSL2
OpenCV version	4.5.4	pkg-config linked
MPI	OpenMPI	Ubuntu 22.04 WSL2
Gaussian kernel	5×5	Stored in constant memory
Blur threshold	variance < 100.0	Laplacian variance metric

Table 3: Parallel Execution Configurations Tested

Configuration	MPI Procs	FPS	Speedup
Sequential (baseline)	1	8.89	1.00×
CPU — MPI + OpenMP (np=1)	1	8.39	0.94×
CPU — MPI + OpenMP (np=2)	2	10.85	1.22×
CPU — MPI + OpenMP (np=4)	4	26.83	3.02×

Configuration	MPI Procs	FPS	Speedup
CPU — MPI + OpenMP (np=8)	8	4.85	0.55×
GPU — CUDA only (np=1)	1	78.42	8.82×
GPU — MPI + CUDA (np=2)	2	27.58	3.10×
GPU — MPI + CUDA (np=4)	4	8.39	0.94×

9 Results and Discussion

9.1 Throughput Analysis

The sequential baseline processed 3921 frames in 440.98 seconds, achieving 8.89 FPS. This is well below the 30 FPS threshold required for real-time operation, confirming that parallelization is not optional but necessary for practical deployment.

The CPU parallel version with 4 MPI processes achieved 26.83 FPS, a $3.02\times$ speedup over sequential. This improvement is attributable to the distribution of frame strips across four independent processes, each applying OpenMP thread-level parallelism within their assigned strip. Performance degraded significantly at 8 processes (4.85 FPS) because the MPI communication overhead for splitting and gathering strips exceeded the computational savings from additional processes on a single machine.

The GPU version with a single MPI process achieved 78.42 FPS, an $8.82\times$ speedup over sequential. This result demonstrates the substantial advantage of pixel-level parallelism for convolution workloads. The GTX 1650 assigns individual threads to individual pixels, processing the entire 1920×1080 frame in a single kernel launch.

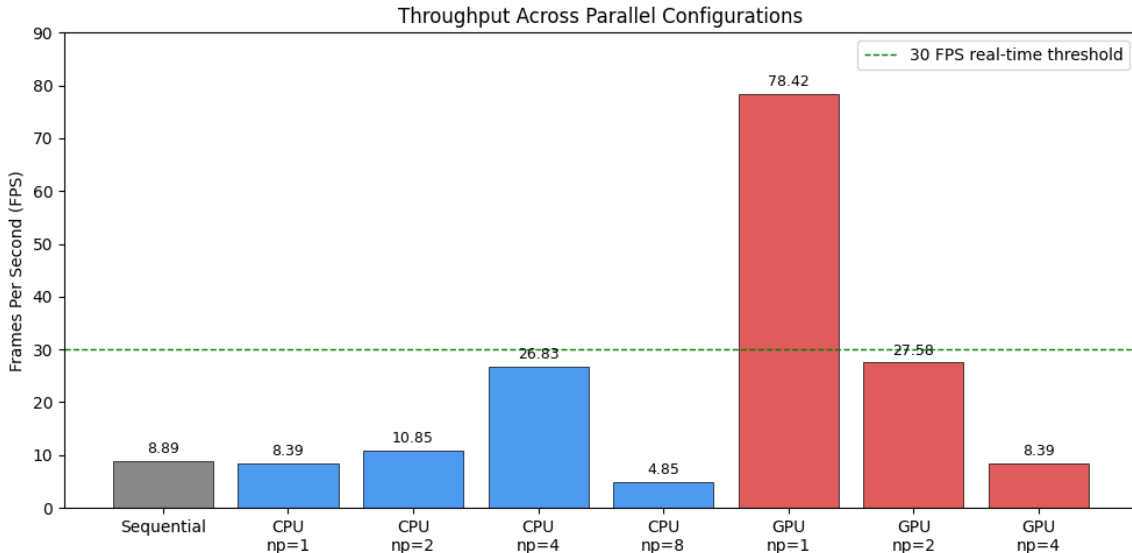


Figure 1: Throughput in frames per second across all parallel configurations

9.2 Effect of MPI Process Count on GPU Performance

A key finding of this project is that increasing MPI process count degraded GPU performance significantly. At 2 MPI processes the GPU achieved 27.58 FPS, and at 4 processes it dropped to 8.39 FPS. This counterintuitive result has two causes. First, all MPI processes share a single physical GPU in this single-node setup, so increasing process count does not increase GPU parallelism but does increase communication overhead. Second, the GTX 1650 operates in WDDM mode through WSL2, which batches GPU command submissions and adds latency to every kernel launch. In a true multi-node cluster where each node has its own GPU, the MPI+CUDA combination would scale linearly.

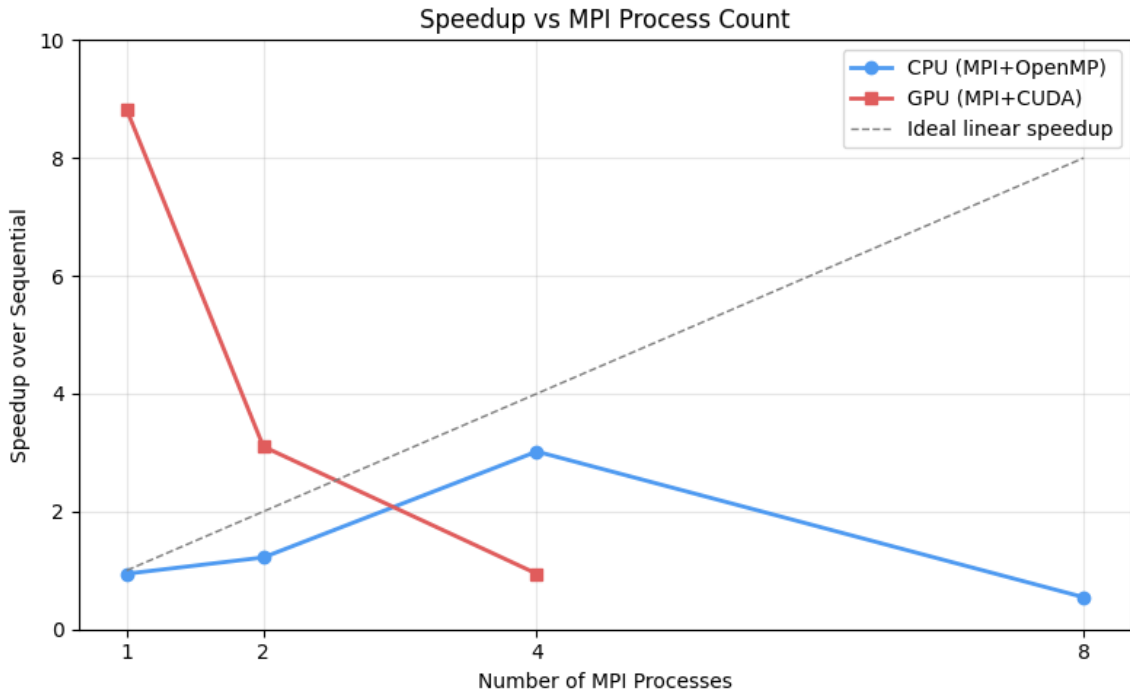


Figure 2: Measured speedup vs number of MPI processes for CPU and GPU versions compared to ideal linear scaling

9.3 Amdahl's Law Analysis

The parallel fraction of the workload can be estimated from the observed speedup. For the CPU version at 4 processes with speedup $S = 3.02$:

$$S(p) = \frac{1}{(1-f) + \frac{f}{p}} \implies f = \frac{\frac{1}{S} - 1}{\frac{1}{p} - 1} = \frac{\frac{1}{3.02} - 1}{\frac{1}{4} - 1} \approx 0.88$$

Approximately 88% of the workload is parallelizable. The remaining 12% comprises MPI initialization, frame reading on rank 0, strip stitching, and video writing, which are inherently serial operations.

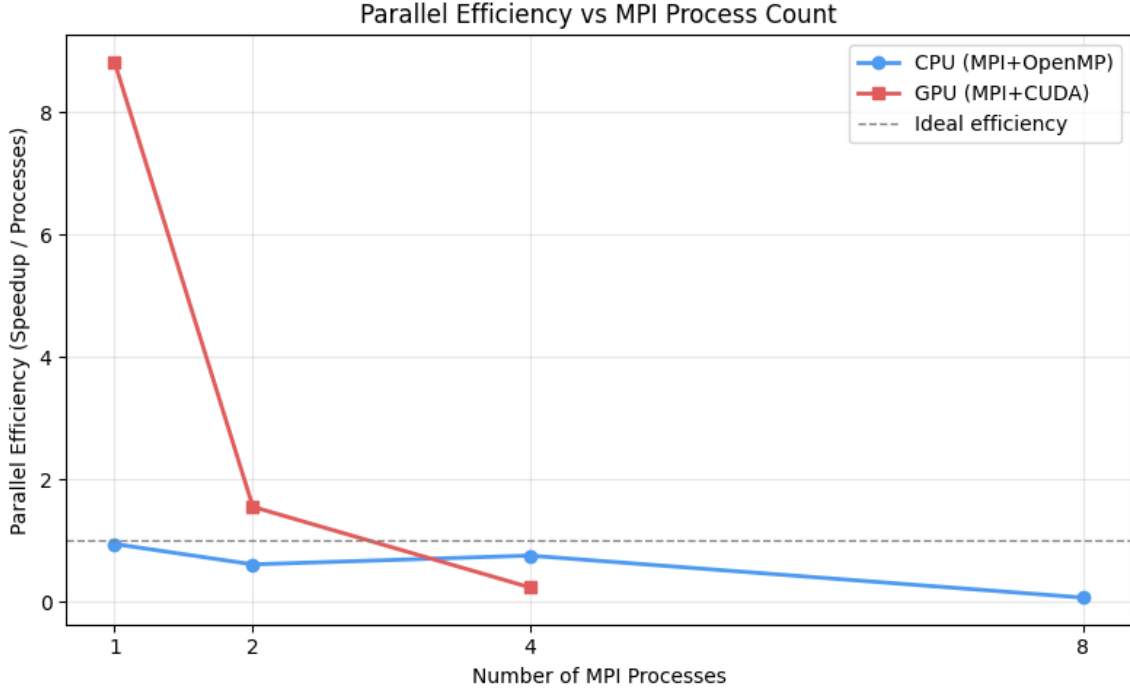


Figure 3: Parallel efficiency vs number of MPI processes for CPU and GPU versions

10 Performance Visualization

To analyze the scalability of the hybrid MPI, OpenMP, and CUDA implementation, three metrics were evaluated across all configurations.

10.1 Execution Time

Figure 1 shows that throughput increases substantially when moving from sequential to parallel execution. The GPU at np=1 achieves the highest throughput at 78.42 FPS, nearly $9\times$ the sequential baseline. The CPU at np=4 achieves the best CPU result at 26.83 FPS. Beyond np=4 for CPU and np=1 for GPU, adding more MPI processes reduces throughput due to communication overhead dominating computation.

10.2 Speedup

Figure 2 compares measured speedup against ideal linear scaling. The CPU version tracks reasonably close to ideal up to np=4, after which overhead causes superlinear degradation. The GPU at np=1 achieves $8.82\times$ speedup, far exceeding what process-level parallelism alone could achieve, because CUDA exploits thousands of hardware threads within a single MPI process.

10.3 Efficiency

Figure 3 shows parallel efficiency defined as $E(p) = S(p)/p$. CPU efficiency at np=4 is approximately 0.75, indicating reasonable utilization with some overhead. GPU efficiency at np=1 exceeds 8.0 relative to the sequential baseline, which reflects the fundamentally different nature of GPU parallelism operating at the thread level rather than the process level.

11 Conclusion

This project implemented and benchmarked a four-stage hybrid parallel video preprocessing pipeline combining MPI, OpenMP, and CUDA. The pipeline performs motion blur detection, Gaussian noise removal, background subtraction, and Sobel edge detection on 1080p surveillance video and is motivated by real-world applications in highway monitoring, retail analytics, and autonomous vehicle preprocessing.

The key findings are:

- Pure CUDA acceleration achieved the highest throughput at 78.42 FPS, an $8.82\times$ speedup over the sequential baseline of 8.89 FPS, demonstrating the effectiveness of pixel-level parallelism for convolution workloads.
- The CPU hybrid (MPI + OpenMP, 4 processes) achieved 26.83 FPS ($3.02\times$ speedup), confirming that process-level distribution provides meaningful gains on multi-core hardware.
- Combining MPI with CUDA on a single node reduced GPU performance due to inter-process communication overhead and WDDM driver batching in WSL2. In a multi-node cluster with one GPU per node, the hybrid MPI+CUDA architecture would be expected to scale linearly with node count.
- Approximately 88% of the pipeline workload is parallelizable, consistent with Amdahl’s Law predictions for the observed CPU speedup. The serial bottleneck is attributable to frame I/O and MPI gather operations on rank 0.

Future work could include porting the pipeline to a CUDA compute mode cluster to properly evaluate MPI+CUDA scaling, extending the pipeline with CUDA-accelerated object detection, and implementing persistent CUDA streams to overlap memory transfers with kernel execution for further latency reduction.

References

- [1] D. Kirk and W.-m. Hwu, *Programming Massively Parallel Processors*. Morgan Kaufmann, 2016.
- [2] J. Sanders and E. Kandrot, *CUDA by Example*. Addison-Wesley, 2010.
- [3] P. Pacheco, *An Introduction to Parallel Programming*. Morgan Kaufmann, 2011.
- [4] G. Bradski and A. Kaehler, *Learning OpenCV*. O'Reilly Media, 2008.